

Fluorometer DOP Explanation

Big-picture architecture (how signals flow)

- Power rails: 12 V (LED driver), 3.3 V (ESP32, ADS1115, logic), GND common.
- UV LED (3 W) powered by the LED driver module (12 V input typical) with driver current set to 350 mA. Driver enable switched by MOSFET or optocoupler under ESP32 control. Heat-sink the LED.
- Blue indicator LED powered from 3.3 V via a series resistor (low current).
- Photodiode (BPW34) connected to a TIA (transimpedance amplifier) implemented with one channel of the MCP602 (zero-bias photodiode configuration). Feedback network provides the three bandwidth/gain bands you designed (2.2 M Ω /150 pF etc.).
- CD4052BE used to select which sensor or path is connected to the amplifier/ADC — place the mux before the TIA input (recommended).
Alternatively use the mux to select which TIA output goes to the ADS1115 if you implement multiple TIAs.
- MCP602 output (buffered) → ADS1115 input(s) via short traces. ADS1115 communicates with ESP32 via I²C.
- ILI9341 SPI TFT connects to the ESP32 SPI pins (MOSI/SCLK/CS/DC/RST) for display.
- ESP32 coordinates LED pulses, mux selection, ADC readings, and UI.

What I presented in the Final Meet

PhotoDiode + OPAMP + ADC + Feedback res and cap + MCU + TFT display screen

Connections

Power rails & decoupling

- **3.3 V** (ESP32 VOUT or external LDO) → powers MCP602, ADS1115, LED (indicator).
- **GND common**: tie all grounds together (ESP32 GND, ADS1115 GND, MCP602 VSS, power supply GND)

MCP602 pinout & recommended wiring (DIP-8)

MCP602 pins (standard):

1: OUT1, 2: IN-1, 3: IN+1, 4: VSS (GND), 5: IN+2, 6: IN-2, 7: OUT2, 8: VDD (3.3 V).

Use **Op-amp 1 (pins 1–3)** as the TIA. Use **Op-amp 2 (pins 5–7)** as a unity buffer for the TIA output.

Wiring:

- Pin 8 → **3.3 V**.
- Pin 4 → **GND**.
- **TIA (Op-Amp 1)**:
 - Photodiode **cathode** → **Pin 2 (IN-1)**.
 - Photodiode **anode** → **GND**.
 - Pin 3 (IN+1) → **GND** (connect to quiet analog ground).
 - **Feedback**: connect **1 MΩ resistor in parallel with 220 pF** between Pin1 (OUT1) and Pin2 (IN-1). Place both components close to the op amp pins.
- **Buffer (Op-Amp 2)**:

- Pin 5 (IN+2) → **Pin1 (OUT1)** (buffer in follower config).
- Pin 6 (IN-2) → **Pin7 (OUT2)**.
- Pin 7 (OUT2) → **ADC input** (through small series resistor; see below).

Notes: Photodiode polarity — above assumes cathode to inverting input and anode to ground so photocurrent produces a negative current into IN-; the op amp will produce a voltage on OUT1 proportional to photocurrent. If you want positive-going output, invert in software or swap polarity/orientation.

ADS1115 connections

- **VCC** → **3.3 V**.
- **GND** → **GND**.
- **SDA** → **ESP32 SDA (GPIO21)**.
- **SCL** → **ESP32 SCL (GPIO22)**.
- **ADDR** → **GND** (address 0x48) unless you want a different I2C address.
- **ALERT/DRDY** not required; leave floating or tie per module docs.
- **Input**: connect ADS1115 AIN0 to MCP602 buffer output (Pin7) **through a 100 Ω series resistor**. That isolates the ADC input cap and protects the op amp from capacitive load. Also place a **100 nF** decoupler near the ADS1115.

ADS1115 PGA suggestion: start with **± 2.048 V** ($G = 1$) or **± 1.024 V** ($G = 2$) depending on expected TIA output. With 1 M Ω : 1 μ A → 1 V, so ± 2.048 V is safe for up to ± 2 μ A.

Expected signal scaling (quick reference)

- With 1 M Ω :
 - 1 μ A $\rightarrow$ 1 V output
 - 100 nA $\rightarrow$ 100 mV
 - 10 nA $\rightarrow$ 10 mV
- ADS1115 LSB (± 2.048 V FS): $4.096 \text{ V} / 2^{16} \approx \mathbf{62.5 \mu V} \rightarrow$ current resolution $\approx \mathbf{62.5 \text{ pA}}$ with 1 M Ω . Good for nA-level signals.

Final checklist before power-up

1. Verify **VDD = 3.3 V** to MCP602 & ADS1115. No 5 V on those pins.
2. Confirm all **grounds tied**.
3. Confirm feedback R (1 M Ω) and C (220 pF NP0) are soldered close to MCP602 pins 1 & 2.
4. Add **100 Ω** series on ADC input and **100 Ω** LED resistor.
5. Start with the LED indicator off / UV LED off. Use low sample rates / test light sources first.
6. If readings look noisy or drifting, check layout, grounding, and add driven guard or reduce R (e.g., to 470 k Ω + 470 pF) if needed.

Interfacing the Screen TFT Display

✓ ILI9341 → ESP32 Wiring (SPI, 10-pin TFT module)

Your module likely has these pins:

TFT Pin	Connect To (ESP32)	Notes	
VCC	3.3V	DO NOT use 5V	
GND	GND	Common ground	
CS	GPIO 5	Chip Select	
RESET	GPIO 4	Reset pin	
DC / RS	GPIO 2	Data/Command	
MOSI	GPIO 23	SPI MOSI	
SCK / CLK	GPIO 18	SPI Clock	
LED	3.3V	Backlight (or PWM pin if dimming)	
MISO	GPIO 19 (optional*)	ILI9341 does not require it for drawing	

* Most ILI9341 modules **don't use MISO**, but if yours has it, connect it to GPIO19.

Libraries Needed

In Arduino IDE:

Sketch → Include Library → Manage Libraries

Install:

- **Adafruit ILI9341**
- **Adafruit GFX**

ESP32 Code,

```
#include <Wire.h>
#include <Adafruit_ADS1X15.h>

#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>

// ----- TFT PINS -----
#define TFT_CS  5
#define TFT_DC  2
#define TFT_RST 4

Adafruit_ILI9341 tft = Adafruit_ILI9341(TFT_CS, TFT_DC, TFT_RST);

// ----- ADS1115 -----
Adafruit_ADS1115 ads;
const adsGain_t gainSetting = GAIN_TWO;

// ----- Graph variables -----
const int graphX = 10;
const int graphY = 140;    // FIXED: moved up for safe zone
const int graphW = 300;
const int graphH = 90;    // FIXED: must keep graph inside screen

int graphPos = 0;
float lastY = -1;

float minVal = 9999;
float maxVal = -9999;

void drawGraphFrame() {
  tft.drawRect(graphX, graphY, graphW, graphH, ILI9341_WHITE);
  tft.setCursor(graphX, graphY - 20);
  tft.setTextColor(ILI9341_CYAN);
  tft.setTextSize(2);
  //tft.print("Signal (Volts)");
}

void setup() {
  Serial.begin(115200);
  delay(500);
```

```

// I2C for ADS1115
Wire.begin(21, 22);

// Initialize ADS1115
ads.begin();
ads.setGain(gainSetting);
ads.setDataRate(RATE_ADS1115_128SPS);
delay(100);

// Initialize TFT
tft.begin();
tft.setRotation(1);      // landscape
tft.fillScreen(ILI9341_BLACK);

// Title
tft.setTextColor(ILI9341_YELLOW);
tft.setTextSize(3);
tft.setCursor(20, 10);
tft.print("Fluorometer");

// Labels
tft.setTextSize(2);
tft.setCursor(10, 60);
tft.print("Voltage (V):");

tft.setCursor(10, 90);
tft.print("Current (nA):");

tft.setCursor(10, 120);
tft.print("Min/Max (V):");

drawGraphFrame();

// Safety clear area at bottom of screen (prevents grey noise)
tft.fillRect(0, graphY + graphH, 320, 20, ILI9341_BLACK);

Serial.println("ADS1115 + TFT Ready.");
}

void loop() {

// ----- Sensing Code (UNCHANGED) -----
int16_t raw = ads.readADC_SingleEnded(0);
float volts = ads.computeVolts(raw);

```

```

float current_A = volts / 1000000.0;
float current_nA = current_A * 1e9;

Serial.printf("ADC: %d | %.6f V | %.3f nA\n",
              raw, volts, current_nA);
// -----

// ----- Update Min/Max -----
if (volts < minVal) minVal = volts;
if (volts > maxVal) maxVal = volts;

// ----- DISPLAY TEXT REFRESH -----

// Voltage
tft.fillRect(150, 60, 150, 25, ILI9341_BLACK);
tft.setTextColor(ILI9341_GREEN);
tft.setTextSize(2);
tft.setCursor(150, 60);
tft.print(volts, 6);

// Current
tft.fillRect(150, 90, 150, 25, ILI9341_BLACK);
tft.setCursor(150, 90);
tft.print(current_nA, 3);

// Min/Max (moved left + wider clear box)
tft.fillRect(100, 120, 200, 25, ILI9341_BLACK);
tft.setCursor(100, 120);
tft.setTextColor(ILI9341_MAGENTA);
tft.print(minVal, 3);
tft.print("/");
tft.print(maxVal, 3);

// ----- GRAPH DRAWING -----

// Erase previous column
tft.drawFastVLine(graphX + graphPos, graphY + 1, graphH - 2, ILI9341_BLACK);

// Map voltage to graph height (0–2.048V)

```



```

float norm = volts / 2.048;
if (norm < 0) norm = 0;
if (norm > 1) norm = 1;

int y = graphY + graphH - (norm * (graphH - 2));

// Draw continuous line
if (lastY < 0) lastY = y;
tft.drawLine(graphX + graphPos - 1, lastY, graphX + graphPos, y, ILI9341_YELLOW);
lastY = y;

// Move graph forward
graphPos++;
if (graphPos >= graphW) {
    graphPos = 1;
    lastY = -1;

    // Clear graph area for next full sweep
    tft.fillRect(graphX + 1, graphY + 1, graphW - 2, graphH - 2, ILI9341_BLACK);
    tft.fillRect(0, graphY + graphH, 320, 20, ILI9341_BLACK); // bottom safety bar
}

delay(150);
}

```

Basically the code plots the real time graph based on the input wavelength and intensity of the light it receives.

Resources

1. ADC Guide: <https://github.com/addicore/ADS1115?tab=readme-ov-file>
2. Photodiode Feedback explanation: <https://www.youtube.com/watch?v=LFTIM07cOFg>
3. Very good Video on Transimpedance Amplifier for Photodiode usage -
 Photodiode amplifier circuit
4. I'll send you datasheets required
5. ESP32 Datasheet: <https://lastminuteengineers.com/esp32-pinout-reference/>